

SIMATS ENGINEERING

ASSESSMENT TOOL 1 (AT1) — COURSE OUTCOME 1 (CO1)

Course Name:	Artificial Intelligence	Course Code:	CSA17
Student Name:	Shaik Reehan	Register Number:	192565071
Duration:	1 Hour	Total Marks:	50 (Weightage: 50%)
Course Outcome:	CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		

Code of Conduct & Student Declaration:

I, **Shaik Reehan (Reg No: 192565071)**, certify that this submission is my original work and that I have adhered to the academic integrity guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of Student: Shaik Reehan

Date: October 8, 2026

ANNEXURE A: ANALYTICAL PROBLEM SOLVING

Question 1: Water Jug Problem Formulation & Solution

Problem Statement: We are given a 4-gallon jug and a 3-gallon jug without measuring marks, and a water pump. The objective is to obtain exactly 2 gallons of water in the 4-gallon jug using allowed operations (fill from pump, empty to ground, pour from one jug into another).

1. Problem Formulation

- State Representation:** An ordered pair (x, y) , where $x \in \{0, 1, 2, 3, 4\}$ denotes water in the 4-gallon jug, and $y \in \{0, 1, 2, 3\}$ denotes water in the 3-gallon jug.
- Initial State:** $(0, 0)$ (both jugs empty).
- Goal State:** $(2, 0)$ or any state $(2, y)$ where the 4-gallon jug contains exactly 2 gallons.

2. Production Rules (Operators)

Rule	Precondition & Action	Rule	Precondition & Action
R1	$(x, y) \rightarrow (4, y)$ if $x < 4$ (Fill 4-gal from pump)	R2	$(x, y) \rightarrow (x, 3)$ if $y < 3$ (Fill 3-gal from pump)
R3	$(x, y) \rightarrow (0, y)$ if $x > 0$ (Empty 4-gal onto ground)	R4	$(x, y) \rightarrow (x, 0)$ if $y > 0$ (Empty 3-gal onto ground)
R5	$(x, y) \rightarrow (4, y - (4 - x))$ if $x + y \geq 4, y > 0$ (Pour 3 $\rightarrow$ 4 till full)	R6	$(x, y) \rightarrow (x + y, 0)$ if $x + y \leq 4, y > 0$ (Pour all 3 $\rightarrow$ 4)
R7	$(x, y) \rightarrow (x - (3 - y), 3)$ if $x + y \geq 3, x > 0$ (Pour 4 $\rightarrow$ 3 till full)	R8	$(x, y) \rightarrow (0, x + y)$ if $x + y \leq 3, x > 0$ (Pour all 4 $\rightarrow$ 3)

3. Step-by-Step Solution Trace

Step	4-Gal Jug (x)	3-Gal Jug (y)	Rule Applied	Operation Description
0	0	0	—	Initial State (both jugs empty)
1	0	3	Rule 2	Fill the 3-gallon jug completely from the pump
2	3	0	Rule 6	Pour all 3 gallons from 3-gal jug into 4-gal jug
3	3	3	Rule 2	Fill the 3-gallon jug completely from the pump again
4	4	2	Rule 5	Pour from 3-gal into 4-gal until 4-gal is full (1 gal poured; 2 gal remain in 3-gal jug)
5	0	2	Rule 3	Empty the 4-gallon jug onto the ground
6	2	0	Rule 6	Pour the remaining 2 gallons from 3-gal jug into 4-gal jug

Result: At Step 6, the 4-gallon jug contains exactly **2 gallons** of water (Goal State $(2, 0)$ reached).

Question 2: Mars Rover Intelligent Agent Analysis

i. Percepts for the Mars Rover Agent

- Stereo Vision & Depth Mapping:** Navcams, Hazcams, and Mastcam capturing RGB images and 3D disparity maps for terrain obstacle detection.

- **Spectrometry & Geochemistry:** ChemCam/SuperCam laser spectroscopy, APXS, and Raman sensors measuring rock/soil elemental chemistry.
- **Kinematics & Pose:** Wheel odometers, 3-axis gyroscopes, and accelerometers in the IMU measuring rover tilt, pitch, roll, speed, heading, and wheel slippage.
- **Mechanical Contact:** Joint force and torque sensors on the robotic arm monitoring resistance during drilling and rock coring.
- **Atmospheric Sensors:** Ambient surface temperature, atmospheric pressure, wind velocity, and dust optical opacity sensors.
- **Uplink Telemetry:** Commands from Earth / Deep Space Network (DSN) providing target coordinates, sol plans, and mission constraints.

ii. Operating Environment Characterization

Dimension	Classification	Reason
Observability	Partially Observable	Cannot see behind terrain obstacles, below the surface, or beyond sensor range; dust storms obstruct cameras.
Determinism	Stochastic	Wheel slip on loose sand is non-deterministic; rock hardness and unexpected mechanical resistance cannot be predicted exactly.
Episodic / Sequential	Sequential	Each driving and drilling choice directly affects subsequent rover position, battery reserves, mechanical wear, and safety.
Static / Dynamic	Dynamic	The environment changes while the rover is computing (sun angle shifts, temperature drops, wind blows dust).
Discrete / Continuous	Continuous	Rover positions, steering angles, motor currents, sensor signals, and time vary continuously.
Single / Multi-Agent	Single-Agent	The rover operates autonomously on Mars without competing artificial agents.

iii. Actions Available to the Agent

- **Locomotion:** Drive forward/reverse, steer individual wheels, turn in place, adjust speed, and trigger emergency stops upon slip/tilt thresholds.
- **Robotic Arm & Sampling:** Articulate robotic arm, position drill/corer, abrade rock surfaces, scoop soil, and store samples in hermetic tubes.
- **Science Operations:** Fire spectrometer lasers, open camera shutters, focus optical lenses, and capture multispectral images.
- **Power & Communications:** Point High-Gain Antenna toward Earth or UHF antenna toward orbiters; control solar panel heaters and enter low-power sleep mode.

iv. Performance Evaluation (Performance Measure)

- **Science Return:** Quantity, geological diversity, and quality of rock core and regolith samples collected and analyzed.
- **Safety & Rover Survival:** Zero fatal entrapments in sand traps; zero mechanical tip-overs; maximizing operational lifetime (sols).
- **Traverse Efficiency:** Distance safely navigated toward target coordinates with minimal path deviation.
- **Energy & Bandwidth Efficiency:** Minimizing energy consumed per meter driven and maximizing scientific data bytes transmitted per uplink window.

v. Most Suitable Agent Architecture & Rationale

Architecture: Utility-Based Agent with a Hybrid (Deliberative + Reactive) Architecture.

Rationale:

1. **Communication Delay:** Earth-to-Mars latency (8 to 44 mins round-trip) rules out real-time teleoperation; the rover must plan autonomously.
2. **Trade-Off Evaluation (Utility Function):** Balances conflicting objectives (e.g., high science value on steep rocks vs. risk of wheel slip and battery drain).
3. **Hybrid Deliberative-Reactive Control:** Deliberative planner schedules path routes, while reactive reflex loops immediately stop motors upon hazard detection.

Question 3: 8-Queens Problem Formulation & Search Strategies

Problem Description: Place 8 queens on an 8×8 chessboard such that no two queens attack each other (no two queens share the same row, column, or diagonal). Note that a board state where the first-column queen shares a diagonal with the last-column queen is non-goal, as diagonal attacks are strictly invalid.

1. Problem Formulation Components

- **State Representation:** An 8-tuple $(r_1, r_2, r_3, r_4, r_5, r_6, r_7, r_8)$, where each $r_i \in \{1, 2, \dots, 8\}$ specifies the row position of the queen in column i . This representation inherently enforces the constraint that each column has exactly one queen.
- **Initial State:** All 8 queens placed arbitrarily in their columns (e.g., $(1, 1, 1, 1, 1, 1, 1, 1)$ or random rows), or an empty board in an incremental formulation.
- **Actions / Operators:** In complete-state formulation, move any single queen in column i from row r_i to another row $r'_i \in \{1, \dots, 8\}$. In incremental formulation, place a queen in an unattacked square in leftmost empty column.
- **Transition Model:** Returns the resulting board state after moving or placing a queen.
- **Goal Test:** 8 queens are placed such that no two queens attack each other. For all pairs of columns (i, j) with $1 \leq i < j \leq 8$:
 1. $r_i \neq r_j$ (no two queens share the same row).
 2. Column uniqueness is guaranteed by the representation.
 3. $|r_i - r_j| \neq |i - j|$ (no two queens share the same diagonal).
- **Path Cost:** Zero. In configuration problems, only the validity of the final state matters, not the intermediate steps.

2. Search Space Size Analysis

Formulation Constraint	Mathematical Formula	Search Space Size
Unconstrained Placement (any 8 of 64 squares)	${}^{64}C_8 = 64! / (8! \times 56!)$	$\approx 4,426,165,368 (\approx 4.4 \times 10^9)$
One Queen per Column (8 choices per column)	8^8	16,777,216
One Queen per Row & Column (Permutations)	$8! = 8 \times 7 \times \dots \times 1$	40,320

3. Search Strategies & Algorithmic Comparison

- **Uninformed Strategy — Depth-First Search (DFS) with Backtracking:** Queens are placed incrementally column-by-column (from column 1 to 8). At column k , the algorithm tests rows 1 to 8, placing the queen on the first square that does not conflict with existing queens. If all 8 rows conflict, it backtracks to column $k-1$ and advances that queen. Since search depth is strictly bounded at 8, DFS with backtracking requires only $O(d)$ space and prunes invalid subtrees early.
- **Informed / Local Search — Min-Conflicts Heuristic:** Starts with all 8 queens on the board. The heuristic function $h(n)$ counts the number of pairs of attacking queens. At each step, a queen that is in conflict is chosen, and moved to the row in its column that minimizes $h(n)$. This local search heuristic resolves the 8-queens problem rapidly, typically in under 50 steps.
- **Constraint Satisfaction Problem (CSP) Formulation:** Variables $X = \{Q_1, Q_2, \dots, Q_8\}$ with domains $D_i = \{1, 2, \dots, 8\}$. Binary constraints specify non-attacking pairs. Using Forward Checking or Arc Consistency (AC-3) combined with the Minimum Remaining Values (MRV) heuristic dramatically reduces search time by pruning conflicting domain values before branching.
- **Breadth-First Search (BFS) Infeasibility:** BFS is unsuitable for the 8-queens problem because its memory requirement grows exponentially ($b^d \approx 8^8 \approx 1.68 \times 10^7$ nodes in the frontier), exceeding practical RAM limits without offering any optimality advantage.

Question 4: Problem-Solving Agent for OLA Cab Booking

Problem Statement: A customer books a cab from a current location to a destination using the OLA mobile app, choosing among cab types (Mini, Micro, Sedan, Shared, Prime) based on comfort, fare, and travel time.

1. Identification & Justification of Problem-Solving Agent Type

Agent Type: Utility-Based Problem-Solving Agent.

Detailed Justification:

- *Simple Reflex Agent:* Maps condition directly to action without memory; cannot handle varying trip locations, dynamic fares, or route calculations.
- *Goal-Based Agent:* Achieves binary goal satisfaction (reaching destination), but treats all valid paths and cab categories as equally acceptable, ignoring trade-offs.
- *Utility-Based Agent (Required):* Customers have conflicting multidimensional preferences: monetary cost, travel comfort, vehicle space, and waiting time (ETA). A utility-based agent employs a scalar utility function $U(s)$ mapping state attributes to real-valued satisfaction scores, ranking available options to select the cab that maximizes expected customer utility.

2. PEAS Framework & Problem Components

- **Performance Measure (P):** High customer satisfaction score, minimized total trip cost and ETA, high booking completion rate.
- **Environment (E):** Urban road network, dynamic traffic conditions, fluctuating cab availability, weather and surge demand.

- **Actuators (A):** Dispatch booking request, display fare/ETA options, push route directions, issue trip invoice.
- **Sensors (S):** GPS receiver, user input touch events, driver telematics, live traffic and map API feeds.
- **State:** $\langle \text{Pickup_Coords}, \text{Drop_Coords}, \text{Available_Fleet}, \text{User_Preferences}, \text{Traffic_State} \rangle$.
- **Mathematical Utility Function:**

$$U(\text{cab}) = w_{\text{comfort}} \times \text{ComfortScore} - w_{\text{fare}} \times \text{EstimatedFare} - w_{\text{time}} \times \text{ETA} + \delta_{\text{pref}}$$

where w_{comfort} , w_{fare} , w_{time} are positive user-defined weights, and δ_{pref} is a bonus applied if the cab matches the user's explicitly selected type.

3. Structured Pseudocode for the OLA Cab Booking Agent

Algorithm: UtilityBasedCabBookingAgent(pickup, drop, user_preferences)
Input: pickup location, drop location, user_preferences (selected_type, max_budget, urgency_weight)
Output: confirmed_booking details OR failure_message

```

1. route ← ComputeOptimalRoute(pickup, drop) // yields route.distance (km) and route.duration (mins)
2. nearby_cabs ← QueryAvailableDrivers(pickup, search_radius = 5.0 km)
3.
4. if nearby_cabs is EMPTY then
5.     return "Error: No cabs currently available in your service area. Please retry shortly."
6. end if
7.
8. evaluated_options ← []
9. for each cab in nearby_cabs do
10.    base_fare ← CalculateBaseFare(cab.type, route.distance, route.duration)
11.    surge_multiplier ← GetSurgeRate(pickup, current_time)
12.    final_fare ← base_fare * surge_multiplier
13.    driver_eta ← CalculateETA(cab.current_coords, pickup)
14.    comfort_rating ← GetComfortScore(cab.type) // Prime: 5, Sedan: 4, Mini: 3, Micro: 2, Shared: 1
15.
16.    if final_fare ≤ user_preferences.max_budget then
17.        utility ← (w_comfort * comfort_rating) - (w_fare * final_fare) - (w_time * driver_eta)
18.        if cab.type == user_preferences.selected_type then
19.            utility ← utility + category_preference_bonus
20.        end if
21.        evaluated_options.append({cab, final_fare, driver_eta, utility})
22.    end if
23. end for
24.
25. if evaluated_options is EMPTY then
26.    return "Notice: No cabs match your specified budget constraint. Consider increasing budget."
27. end if
28.
29. Sort evaluated_options in DESCENDING order of utility
30. best_candidate ← evaluated_options[0]
31.
32. dispatch_status ← DispatchRideRequest(best_candidate.cab, pickup, drop, best_candidate.final_fare)
33. if dispatch_status == ACCEPTED then
34.    return ConfirmBooking(best_candidate.cab, best_candidate.driver_eta, best_candidate.final_fare)
35. else
36.    // Fallback: Sequentially query next best candidate options
37.    return HandleDriverRejectionFallback(evaluated_options[1:])
38. end if

```

4. Edge Case & Exception Handling

If a driver cancels or times out (after 15 seconds), the agent initiates the fallback procedure, querying the next candidate in *evaluated_options*. During peak hours, the agent dynamically suggests alternative cab categories within budget that have lower wait times.

Question 5: Uniform Cost Search (UCS) on Delivery Logistics Network

Problem Statement: A logistics company needs to transport packages between warehouses at minimum cost. The network is modeled as a weighted directed graph where each edge represents travel cost (fuel + time). Determine the least-cost path from start warehouse **S** to destination warehouse **G** using the Uniform Cost Search (UCS) algorithm.

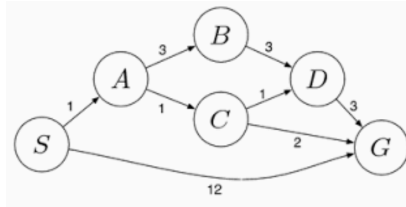


Figure 5.1: Weighted graph representation of the delivery network (Warehouses S to G)

1. Graph Specifications & UCS Principles

- **Start Node:** S | **Goal Node:** G
- **Directed Edges & Step Costs:** $S \rightarrow A : 1$, $S \rightarrow G : 12$, $A \rightarrow B : 3$, $A \rightarrow C : 1$, $B \rightarrow D : 3$, $C \rightarrow D : 1$, $C \rightarrow G : 2$, $D \rightarrow G : 3$.
- **UCS Mechanics:** UCS expands nodes in order of non-decreasing cumulative path cost $g(n)$ using a Priority Queue. An *Explored Set* prevents cycles. The goal test is applied **only when a node is dequeued from the priority queue**, which guarantees finding the optimal (least-cost) path.

2. Step-by-Step UCS Execution Trace

Iter.	Dequeued Node	Current Path	Cost g	Successors Generated	Priority Queue / Frontier (Node, Cost, Path)
0	—	—	0	Initialize start node S	[(S, 0, S)]
1	S	S	0	A (0 + 1 = 1), G (0 + 12 = 12)	[(A, 1, S-A), (G, 12, S-G)]
2	A	S → A	1	B (1 + 3 = 4), C (1 + 1 = 2)	[(C, 2, S-A-C), (B, 4, S-A-B), (G, 12, S-G)]
3	C	S → A → C	2	D (2 + 1 = 3), G (2 + 2 = 4)	[(D, 3, S-A-C-D), (B, 4, S-A-B), (G, 4, S-A-C-G), (G, 12, S-G)]
4	D	S → A → C → D	3	G (3 + 3 = 6)	[(B, 4, S-A-B), (G, 4, S-A-C-G), (G, 6, S-A-C-D-G), (G, 12, S-G)]
5	B	S → A → B	4	D (4 + 3 = 7, discarded as D already explored)	[(G, 4, S-A-C-G), (G, 6, S-A-C-D-G), (G, 12, S-G)]
6	G	S → A → C → G	4	Goal reached! Terminate search.	—

Note on tie-break at step 5: Both node B and node G have cumulative cost $g = 4$. By standard convention (alphabetical priority or expanding non-goal nodes first), B is dequeued first. Expanding B generates D with cost 7, which is discarded since D is already explored with cost 3. Node G is dequeued next at cost 4, passing the goal test and terminating the search.

3. Verification of All Feasible Paths from S to G

Path #	Route Taken	Calculation (Sum of Edge Costs)	Total Cost
1	S → G (direct route)	12	12
2	S → A → B → D → G	1 + 3 + 3 + 3	10
3	S → A → C → D → G	1 + 1 + 1 + 3	6
4	S → A → C → G (Optimal)	1 + 1 + 2	4

4. Final Result & Engineering Interpretation

Least-Cost Delivery Path: $S \rightarrow A \rightarrow C \rightarrow G$ with a **Minimum Travel Cost of 4** units (fuel + time). This route reduces the delivery cost by 66.7% compared to the direct link $S \rightarrow G$ (cost 12), demonstrating the operational efficiency of Uniform Cost Search in supply chain routing.